



Questions carrying 2 marks:

- 1) What is stack? Name the basic operations performed on stack.
- 2) What is LIFO list? Mention any 2 its applications.
- 3) Mention any two applications of stack.
- 4) What do you mean by PUSH and POP operation?
- 5) Expand LIFO and FIFO.
- 6) What is a queue? Why queue is called FIFO list?
- 7) How does Stack differ from Queue?
- 8) Write an algorithm for PUSH operation using arrays.
- 9) Write an algorithm for POP operation using arrays.
- 10) Write an algorithm to find factorial of a number N using recursion.
- 11) Write an algorithm to generate Fibonacci series using recursion.
- 12) Define i) infix notation ii) prefix notation iii) postfix notation.

Page 18 of 25

- 13) What do you mean by infix notation? Give an example.
- 14) What do you mean by prefix notation? Give an example.
- 15) What do you mean by postfix notation? Give an example.
- 16) What do you mean by polish and reverse polish notation?
- 17) Define queue. Name the primary queue operations.
- 18) Differentiate queue and circular queue.
- 19) What is circular queue?
- 20) Define dequeue. What are its types?
- 21) What do you mean by input-restricted deque and output-restricted deque.
- 22) What is meant by priority queue? What is its use?

Long Answer Questions

- 1) Write an algorithm for PUSH and POP operations using arrays. (4)
- 2) Write an algorithm to push and POP operation onto a stack using linked list. (6)
- 3) Convert the given infix expression to prefix and postfix form.(any 2) (6)
 - a) $A * B + (C \wedge D / E / F)$
 - b) $a / b * c - d + e / f * (g + h)$
 - c) $(X + Y / Z * W \wedge P) - R$
 - d) $(X + Y * Z \wedge A / 4) - P$
 - e) $(A + B) * (C - D) / E * F$
 - f) $a * b + (c / d \wedge e) - f$
 - g) $(A - B * C \wedge D) / (E + F)$
 - h) $(a / b) * (c * f + (a - d) * e)$
 - i) $A + B * (C - D * (E + F))$
 - j) $a * b - c \wedge d + e / f$
- 4) Write an algorithm to evaluate postfix expression using stack(4)
- 5) Write an algorithm to convert infix to postfix expression using stack.(6)
- 6) Convert the following infix expression into postfix expression using stack status. Q: $(A + (B * C - (D / E \wedge F) * G) * H)$ (6)
- 7) Convert the following infix expression into postfix expression using stack status. Q: $((A + B) * D) \wedge (E - F)$ (6)
- 8) Evaluate the following postfix expression showing the stack status. P: 3, 5, +, 6, 4, -, *, 4, 1, -, 2, ^, + (6)
- 9) Evaluate the following postfix expression showing the stack status. P: 3, 1, +, 2, ^, 7, 4, -, 2, *, +, 5, - (6)
- 10) Evaluate the following postfix expression showing the stack status. P: 5, 6, 2, +, *, 1, 2, 4, /, ^ (6)
- 11) What is a Queue ? Explain the different types of queue. (6)
- 12) Write an algorithm to perform insertion operation on queue using arrays. (4)
- 13) Write an algorithm to perform deletion operation on queue using arrays. (4)
- 14) Write an algorithm to perform insertion operation on queue using linked list. (4)
- 15) Write an algorithm to perform deletion operation on queue using linked list (4)
- 16) Write an algorithm to perform insertion operation on circular queue using array (4)
- 17) Write an algorithm to perform deletion operation on circular queue using array (4)
- 18) Write a note on i) circular queue. ii) Dequeue. (4)
- 19) What is Priority queues? What are the types ?Explain (4)

Unit-3

2M 1) what is stack name the 2 operation performed on stack..

It is ordered collection of elements, where insertion & deletion of element take place at the same end that end is called TOP end.

- (1) PUSH (2) POP (3) Display

2) what is LIFO list? Mention any 2 its application

The deletion & insertion in a stack is done from top of the stack, the last added element will be removed first from the stack. So stack is also called LIFO - Last In First Out list.

> Used in many algorithms like Tower of Hanoi, tree traversals.

3) Mention any 2 applications of stack

> used for Backtracking.

> " " Systematic memory management.

> used to convert one form of expression to another form.

4) what do you mean by PUSH & POP operation?

PUSH: The process of adding a new element to the top of the stack is called PUSH operation.

POP: The process of deleting an element from the top of the stack is called POP operation.

5) Expand LIFO & FIFO

LIFO → Last In First Out

FIFO → First In First Out

6) what is Queue? why queue is called FIFO list?

Queue is a non-primitive linear DS, where insertion of an element takes place at rear end and deletion of element takes place at front end.

The first added element is removed from the list. Therefore Queue is called FIFO list

7. How does stack differ from Queue.
* In stack Insertion & deletion of an element takes place in same end called Top end.
But in Queue Insertion at rear end & deletion at front end.
* Stack follows LIFO list while Queue follows FIFO List.

8) write an algorithm for PUSH operation using array.

step 1: Insertion(a, top, item, max)

step 2: If $top = max$ then

print "Stack Overflow"

exit

else

step 3: $top = top + 1$

end if

step 4: $a[top] = item$

step 5: exit

9. Write an algorithm for POP operation using array

step 1: Deletion(a, top, item)

step 2: if $top = 0$ then

print "Stack UNDERFLOW"

exit

else

step 3: $item = a[top]$

end if

step 4: $top = top - 1$

exit

- 2) Define (i) infix notation (ii) prefix notation (iii) postfix notation
- In an expression, if an operator is in between 2 operands then it's called infix notation.
 - In an expression, if an operator comes before the 2 operands, then it's called prefix notation.
 - In an expression, if an operator comes after the 2 operands, then it's called postfix notation.

13) What is infix notation? Give an ex.

14) " " prefix " "

15) " " postfix " "

13) ex: $(2+3) \times 5 = 5 \times 5 = 25$

$2 + (3 \times 5) = 2 + 15 = 17$

14) ex: $2 + 3 \times 5 = 2 + 3 \times 5$

$2 + 15 = 17$

$2 \times 3 + 5 = 2 \times 5 + 5 = 25$

15) ex: $2 \times 3 + 5 = 2 \times 5 + 5 = 17$

$2 \times 3 + 5 = 5 \times 5 = 25$

16. What do you mean by polish & reverse polish notation?

polish:- polish notation reverse the notation in which operator symbol is placed before its 2 operands.

reverse:- It refers the notation in which operator symbol is placed after its 2 operands.

17. Define Queue: Name the primary ^{Queue} operations.

~~Queue~~ See ans for Q.6. - Definition

* Insert

* Delete

* Traverse

18. Differentiate Queue & Circular Queue.

Queue

* It stores the data as sequence of element

* possible to enter a new items to rear & delete from front.

* Requires more memory

* less efficient

Circular Queue

> In this last item connects back to the first item forming a circle

> possible to enter and remove elements from any position

> Requires less memory

> more efficient

19. What is Circular Queue.

Circular Queue is the extended version of a regular queue where the last element is connected to the first element. Thus forming a circle, where insertion & deletion can be performed on any position.

20. Define deque? What are its types

Deque or double ended queue is a type of queue in which insertion & deletion of element can either be performed from the front or the rear.

Types (i) Input-restricted deque

(ii) Output-restricted deque

21. What is Input-restricted deque & output-restricted deque.

→ In Input-restricted deque deletion can be made from both ends, but insertion can be made at one end only

→ In output-restricted deque insertion can be made at both end & deletion can be made from only one end

20) what is meant by priority queue? what is its use?
A priority queue is a collection of elements where the elements are stored ^{accordi} according to their priority levels. adding or removal of element is also decided by the priority of element.
Use: used for implementing job scheduling where jobs with higher priority proceed first.

10. write an algorithm to find factorial of a number N using recursion.

Factorial (fact N)

step 1: if $N=0$ then set fact = 1 & return

step 2: Set fact = 1

step 3: Repeat for $k=1$ to N

set fact = $k \times$ fact

(End of loop)

step 4: Return

11. write an algorithm to generate fibonacci series using recursion.

step 1: if $N=0$ or $N=1$ then set FIB = N & return

step 2: call FIBONACCI(FIB A, $N-2$)

step 3: call FIBONACCI(FIB B, $N-1$)

step 4: Set FIB = FIB A + FIB B

step 5: Return.

8) write an algorithm for PUSH operation using array.

step 1: Insertion(a, top, item, max)

step 2: If $top = max$ then

print "Stack Overflow"

exit

else

step 3: $top = top + 1$

end if

step 4: $a[top] = item$

step 5: exit

9. Write an algorithm for POP operation using array

step 1: Deletion(a, top, item)

step 2: if $top = 0$ then

print "Stack UNDERFLOW"

exit

else

step 3: $item = a[top]$

end if

step 4: $top = top - 1$

exit

Date ___/___/___

Algorithm for evaluating a postfix expression.

> Let P is an expression written in postfix notation.

> ① Stack = empty stack

② Scan P from left to right and repeat steps and 4 for each symbol in P until end of expression

③ If an operand is encountered, push it on stack

④ If an operator is encountered then;

> a) operand 2 = pop(stack)

b) operand 1 = pop(stack)

c) value = operand 1 $\times$ operand 2

d) push value on stack

⑤ Return the value at top of the stack.

⑥ Exit.

Algorithm

1. Read the infix expression from left to right one character at a time.
2. Repeat step 3
3. Read symbol
 - a. If symbol is operand put into postfix string
 - b. If symbol is left parenthesis push into stack
 - c. If right parenthesis, pop stack of TOP until left parenthesis occur and make postfix expression
 - d. If operator then
 - I. If operator has same or less precedence than operators available at top of stack, then pop all such operators and form postfix string
 - II. Push scanned /incoming operator to the stack
4. Until last of string encountered
5. Pop all the element from stack to make stack empty

Q. 6. Convert the following infix expression into postfix expression using stack.

$$(A + (B * C - (D / E * F) * G) * H)$$

Symbol	stack	postfix expression
(	(	
A	(	A
+	(+	A +
(	((+	A +
B	((+	AB
*	((+ *	AB *
C	((+ *	ABC
-	((+ * - ^{POP *}	ABC*
(	(((+ -	ABC*
D	(((+ -	ABC*D
/	(((+ - /	ABC*D/
E	(((+ - /	ABC*DE
*	(((+ - / *	ABC*DE
F	(((+ - / *	ABC*DEF
)	(((+ - / *)	ABC*DEF/
*	(((+ - * /	ABC*DEF/
G	(((+ - * /	ABC*DEF/G
)	(((+ - * / *)	ABC*DEF/G*
*	(((+ * / *)	ABC*DEF/G*
H	(((+ * / *)	ABC*DEF/G*H
)	(((+ * / *) +	ABC*DEF/G*H*

7. Q. $(A + B) * D \wedge (E - F)$

Symbol	stack	postfix expression
(	(	
(	((	
A	((	A
+	((+	A +
B	((+	AB
)	((+)	AB +

*	(*	AB+
D	(*	AB+D
)	(*)	AB+D*
^	^	AB+D*
(	^(	AB+D*
E	^(	AB+D*E
-	^(-	AB+D*E
F	^(-	AB+D*EF
)	^(-)	AB+D*EF-1

8. Evaluate the following postfix expression showing the stack status.

Q: 3, 1, +, 2, ^, 7, 4, -, 2, *, +, 5, -

Sym	opnd1	opnd2	value	stack
3				3
1				3, 1
+	3	1	4	4
2				4, 2
^	2	4	16	16
7				16, 7
4				16, 7, 4
-	7	4	3	16, 3
2				16, 3, 2
*	3	2	6	16, 6
+	16	6	22	22
5				22, 5
-	22	5	17	<u>17</u>

0: 3, 5, +, 6, 4, -, *, 4, 1, -, 2, \$, +

12. Evaluate the following postfix expression showing the stack status.

sym	opnd1	opnd2	value	stack
3				3
5				3, 5
+	3	5	3+5 8	8
6				8, 6
4				8, 6, 4
-	6	4	2	8, 2
*	8	2	16	16
4				16, 4
1				16, 4, 1
-	4	1	3	16, 3
2				16, 3, 2
\$	3	2	9	16, 9
+	16	9	25	25

QUEUE

Queue is a non-primitive data structure.
It is a ordered collection of homogeneous elements. It follows FIFO (First In First Out).
Insert end $\rightarrow$ rear end
Delete end $\rightarrow$ front end
Total no. of elements = $\text{rear} - \text{front} + 1$

Operations

Insertion (Enqueue)

Simple Queue

In a simple queue, insertion takes place at the rear and removal occurs at the front. It strictly follows the FIFO (First in First out) rule.



Deque (Double Ended Queue)

In a double ended queue, insertion and removal of elements can be performed from either from the front or rear. Thus, it does not follow the FIFO (First In First Out) rule.

A priority queue is a special type of queue in which each element is associated with a priority and is served according to its priority. If elements with the same priority occur, they are served according to their order in the queue.

9. What is Circular Queue.

Circular Queue is the extended version of a regular queue where the last element is connected to the first element. Thus forming a circle, where insertion & deletion can be performed on any position.

3. Write an algorithm to insert/delete an element into a queue using arrays. (5)

* Insertion (Enqueue)

step 1: if rear = max - 1 then
write "Queue is overflow"
Go to step 4
[End if]

step 2: if front = -1 & rear = -1
set front = rear = 0
else

set rear = rear + 1
[End if]

step 3: set Queue[rear] = num

step 4: Exit

* deletion (Dequeue)

step 1: if front = -1 or front > rear
write "Queue is underflow"

else

set val = Queue[front]
front = front + 1
[End if]

step 2: Exit

ALGORITHMS FOR INSERTION & DELETION IN A LINEAR QUEUE DYNAMIC IMPLEMENTATION USING LINKED LIST

(1) Algorithm for inserting an element in a Linear Queue :

Let PTR is the structure pointer which allocates memory for the new node & element to be inserted into linear queue, INFO represents the information part of LINK represents the link or next pointer pointing to the address of next node. FRONT represents the address of first node, REAR represents the address of the last node. Before inserting first element in the queue, FRONT=REAR=NULL.

Step 1 : Allocate memory for the new node using PTR.

Step 2 : Read NUM to be inserted into linear queue.

Step 3 : Set PTR->INFO = NUM

Step 4 : Set PTR->LINK= NULL

Step 5 : If FRONT = NULL : then

Set FRONT=REAR=PTR

Else

Set REAR->LINK=PTR;

Set REAR=PTR;

[End of If Else Structure]

Step 6 : Exit

(2) Algorithm for Deleting a node from a Linear Queue :

Let PTR is the structure pointer which deallocates memory of the first node of linear queue & NUM is the element to be deleted from queue, INFO represents information part of the deleted node and LINK represents the link or next pointer of deleted node pointing to the address of next node. FRONT represents the address of first node, REAR represents the address of the last node.

Step 1 : If FRONT = NULL : then

Write 'Queue is Empty(Queue Underflow)' and return.

[End of If structure]

Step 2 : Set PTR = FRONT

Step 3 : Set NUM = PTR->INFO

Step 4 : Write 'Deleted element from linear queue is : ', NUM.

Step 5 : Set FRONT = FRONT->LINK

Step 6 : If FRONT = NULL : then

Set REAR = NULL.

[End of If Structure].

Step 7 : Deallocate memory of the node at the beginning of queue using PTR.

Step 8 : Exit.

6) write an algorithm to insert/delete a node following a circular queue (6)

Insertion:

~~step 1: if $FRONT = (REAR + 1) \text{ Mod } N$ and~~

step 1: if $front = -1$ and $rear = -1$ then set $front = 0$ and goto step 4

step 2: if $front = 0$ and $rear = N-1$ or $front = (rear + 1) \text{ Mod } N$

Mod N

then print: "Circular Queue Overflow" and Return

step 3: if $rear = N-1$ then set $rear = (rear + 1) \text{ Mod } N$ and goto step 5

step 4: set $rear = (rear + 1) \text{ Mod } N$

step 5: set $queue[rear] = item$

step 6: Return

Deletion:

step 1: if $front = -1$ then

print: "Circular Queue Underflow" and Return

step 2: set $item = queue[front]$

step 3: if $front = N-1$ then set $front = (front + 1) \text{ Mod } N$ and Return

step 4: if $front = rear$ then set $front = -1$ and $rear = -1$ and Return

step 5: set $front = (front + 1) \text{ Mod } N$

step 6: Return

Queue

* It stores the data as sequence of element

* possible to enter a new items to rear & delete from front.

* Requires more memory

* less efficient

Circular Queue

> In this last item connects back to the first item forming a circle

> possible to enter and remove elements from any position

> Requires less memory

> more efficient

19. What is Circular Queue.

Circular Queue is the extended version of a regular queue where the last element is connected to the first element. Thus forming a circle, where insertion & deletion can be performed on any position.

20. Define deque? What are its types?

Deque or double ended queue is a type of queue in which insertion & deletion of element can either be performed from the front or the rear.

Types (i) Input-restricted deque

(ii) Output-restricted deque

21. What is Input-restricted deque & output restricted deque.

→ In Input-restricted deque deletion can be made from both ends, but insertion can be made at one end only

→ In output-restricted deque insertion can be made at both end & deletion can be made from only one end

CIRCULAR QUEUES

The queue that we implemented using an array suffers from one limitation. In that implementation there is a possibility that the queue is reported as full (since rear has reached the end of the array), even though in actuality there might be empty slots at the beginning of the queue. To overcome this limitation we can implement the queue as a circular queue. Here as we go on adding elements to the queue and reach the end of the array, the next element is stored in the first slot the array (provided it is free). Suppose an array **arr** of **n** elements is used to implement a circular queue we may reach **arr[n-1]**. We cannot add any more elements to the queue since we have reached at the end of the array. Instead of reporting the queue as full, if some elements in the queue have been deleted then there might be empty slots at the beginning of the queue. In such a case these slots would be filled by new elements being added to the queue. In short just because we have reached the end of the array, the queue would not be reported as full.

The queue would be reported as full only when all the slots in the array stand occupied. Figure (4) shows the pictorial representation of a circular queue.

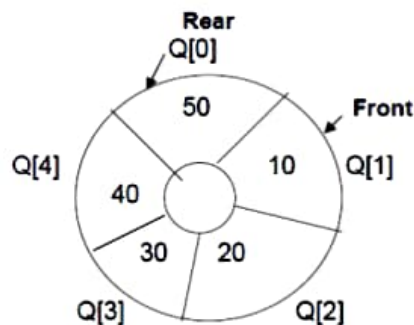


Fig. (4) : Pictorial representation of a circular queue

last location of queue then we can insert a new element at the beginning of the array.

PRIORITY QUEUE

A **priority queue** is a collection of elements where the elements are stored according to

their priority levels. The order in which the elements get added or removed is decided by the priority of the element.

Following **rules** are applied to maintain a priority queue :

- (1) The element with a higher priority is processed before any element of lower priority.
- (2) If there are elements with the same priority, then the element added first in the queue would get processed.

Priority queues are used for implementing job scheduling by the operating system where jobs with higher priorities are to be processed first. Another application of Priority queues is simulation systems where priority corresponds to event times.

There are mainly two ways of maintaining a priority queue in memory. One uses a oneway list, and the other uses multiple queues. The ease or difficulty in adding elements to or deleting them from a priority queue clearly depends on the representation that one chooses.

Ascending Order Priority Queue

An ascending order priority queue gives the highest priority to the lower number in that queue. For example, you have six numbers in the priority queue that are 4, 8, 12, 45, 35, 20.

Firstly, you will arrange these numbers in ascending order. The new list is as follows: 4, 8, 12, 20, 35, 45. In this list, 4 is the smallest number. Hence, the ascending order priority queue treats number 4 as the highest priority.

4	8	12	20	35	45
----------	----------	-----------	-----------	-----------	-----------

In the above table, 4 has the highest priority, and 45 has the lowest priority.

Descending Order Priority Queue

A descending order priority queue gives the highest priority to the highest number in that queue. For example, you have six numbers in the priority queue that are 4, 8, 12, 45, 35, 20.

Firstly, you will arrange these numbers in ascending order. The new list is as follows: 45, 35, 20, 12, 8, 4. In this list, 45 is the highest number. Hence, the descending order priority queue treats number 45 as the highest priority.

45	35	20	12	8	4
-----------	-----------	-----------	-----------	----------	----------

In the above table, 4 has the lowest priority, and 45 has the highest priority.